

SUMMER INTERNSHIP REPORT

FairAC: A Decentralized IoT and Mobile Ecosystem for Fair Air Conditioning Billing in Shared Accommodations

Submitted in partial fulfilment of the requirements of the M.Sc. (Information Technology) Programme

Student Name	[Your Full Name]
Enrollment No.	[Enrollment Number]
Programme	M.Sc. (Information Technology), DA-IICT
Company Name	[Company Name]
Company Address	[Company Address]
Mentor / HR	[Mentor Name]
Internship Duration	[Start Date] to [End Date] ([Duration])

Table of Contents

1. Introduction	3
2. Context Diagram	4
3. Scope Statement	5
4. Detailed Use Cases Implemented	6
5. Design Contribution	7
6. Programming Contribution	8
7. Tools, Technologies, APIs and Libraries Used	9
8. Testing Strategies and Reports	10
9. Innovative Contribution	11
10. Lessons Learnt	12
References	13

1. Introduction

1.1 Background

Air conditioners are among the most power-intensive appliances found in shared hostel and PG accommodations. The prevailing convention in most such rooms is to divide the total monthly electricity bill equally among all occupants, irrespective of how much each individual actually used the appliance. While simple to administer, this approach does not account for genuine differences in usage patterns among roommates.

1.2 Problem Statement

Equal-division billing leads to a persistent lack of transparency and fairness: occupants who use the air conditioner sparingly end up subsidising those who use it heavily, and there is no reliable, tamper-proof record of who consumed how much energy and for how long. This gap in transparency is a recurring source of disputes in shared accommodation.

1.3 Objectives

- To design an IoT hardware module capable of metering high-power appliances such as air conditioners with accuracy and reliability.
- To develop a secure, offline-first communication protocol between the mobile application and the hardware module using Bluetooth Low Energy (BLE), so that the system remains usable even without an active internet connection.
- To build a transaction-safe backend capable of handling digital wallets and proportional billing splits among multiple participants.

1.4 Existing Systems and Literature Review

Two broad categories of existing solutions were studied before arriving at the FairAC design:

- **Traditional sub-metering:** Installing a dedicated electrical sub-meter per room is accurate but expensive, and typically requires rewiring the building — making it impractical for existing hostel infrastructure.
- **Smart plugs:** Commercially available smart plugs are easier to install but generally require a constant Wi-Fi connection and are not rated to handle the high-amperage loads (16A and above) drawn by air conditioners.

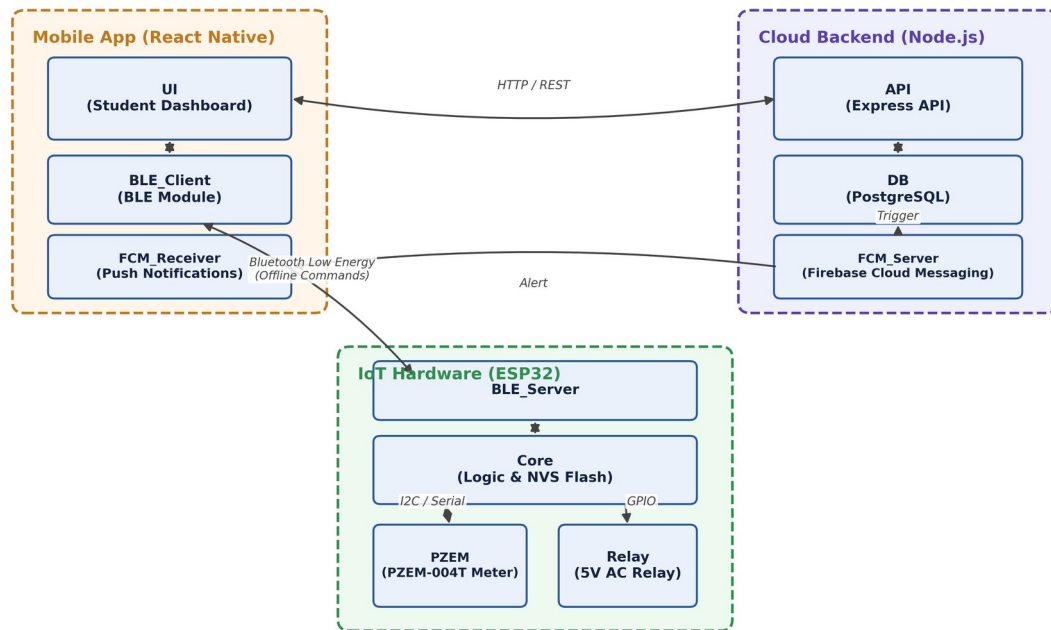
FairAC was designed to bridge both gaps by combining a robust PZEM-004T current/voltage sensing module with an offline-capable BLE communication protocol, avoiding the cost of rewiring while still supporting high-amperage AC loads reliably.

1.5 Proposed Solution and Key Outcomes

The proposed solution is a full-stack system that integrates an IoT hardware module (built around the ESP32 microcontroller) for precise, real-time energy monitoring, a React Native mobile application for decentralized session control, and a PostgreSQL-backed cloud service for exact, time-proportional split-billing. Over the course of development, the system was demonstrated to support offline-first execution during network outages, recover gracefully from mid-session power cuts, and compute mathematically fair, second-level accurate cost distribution among all participants of a shared AC session.

2. Context Diagram

FairAC — System Context / Component Diagram



The student is the primary actor in the ecosystem and interacts with the system entirely through the Mobile App. The Mobile App communicates with the ESP32 Hardware over Bluetooth Low Energy when offline, and with the Cloud Backend over HTTPS when an internet connection is available. On the hardware side, the ESP32 controls the Physical AC through a relay and continuously measures live power consumption using the PZEM-004T sensor. On the cloud side, the Backend manages wallets, sessions, and billing records in a PostgreSQL database, and dispatches real-time alerts to users through Firebase Cloud Messaging (FCM).

3. Scope Statement

3.1 In-Scope (Built During This Internship)

- IoT hardware module: ESP32 + PZEM-004T + Relay + Emergency Stop button + LCD display.
- ESP32 firmware (C++): full offline BLE session management, NVS-based power-cut recovery, and autonomous energy tracking.
- Backend REST API (Node.js / Express): authentication, wallets, sessions, billing, and push notifications.
- PostgreSQL database: schema design, ACID-compliant transactions, and financial safety constraints.
- Student mobile app (React Native / Expo): dashboard, BLE control, wallet, and session history.

3.2 Out-of-Scope (Planned for Future Work)

- Web-based admin dashboard for hostel wardens.
- iOS version of the mobile app.
- Actionable push-notification buttons (Accept / Reject directly from the lock screen).

4. Detailed Use Cases Implemented

UC-01: Student Books an AC Session

Actor: Student

Precondition: Student is logged in, has a wallet balance of at least ₹50, and is an assigned member of the room.

Flow: Student selects a booking type (Duration / Units / Amount) → funds are blocked in the wallet → the app connects to the ESP32 over BLE → a START command is sent.

Postcondition: The AC relay activates and the ESP32 begins tracking energy consumption autonomously.

UC-02: Roommate Joins a Session (Split Billing)

Actor: Roommate

Precondition: An active AC session already exists in their room.

Flow: The session creator invites the roommate → the roommate receives a push notification → the roommate accepts → a joined_at timestamp is recorded → the final bill is split proportionally based on time spent.

Postcondition: Both participants are billed in proportion to their actual time in the session.

UC-03: Session Auto-Terminates (Limit Reached)

Actor: ESP32 (Automated)

Flow: The ESP32's kWh counter crosses the configured max_kwh limit → the relay is shut off → the ESP32 broadcasts a FINISHED status over BLE → the mobile app receives the status and sends the final kWh reading to the server → the server calculates and deducts the exact cost from all participants and refunds any unused blocked amount to the creator.

UC-04: Hardware Emergency Stop

Actor: Physical button on the hardware

Flow: The student presses the emergency-stop button → the ESP32 immediately shuts off the relay → the session state is saved to NVS flash → a FINISHED broadcast is sent over BLE → the app auto-syncs the session with the server.

UC-05: Power-Cut Recovery

Actor: ESP32 (Automated)

Flow: Power is restored → the ESP32 reads the saved session_kwh and remaining_duration values from NVS → the session automatically resumes and the countdown continues from where it left off.

UC-06: Student Wallet Top-Up and History

Actor: Student

Flow: The student enters a top-up amount → the wallet balance is credited → the student can then view the full transaction history with type filters for Recharge and Consumption entries.

5. Design Contribution

5.1 System Architecture

I designed a three-tier, offline-first architecture consisting of an IoT Edge Layer, a Mobile Bridge Layer, and a Cloud Backend Layer. This design ensures that the system continues to function correctly even during complete network outages, with the mobile app acting as a secure bridge between the offline hardware and the online backend.

5.2 Database Schema Design

I designed a normalized set of PostgreSQL tables — users, hostels, rooms, room_members, wallets, wallet_transactions, sessions, session_participants, and consumption_records — along with financial safety constraints such as CHECK (units_consumed >= 0) and CHECK (cost >= 0) to prevent invalid financial states from ever being written to the database.

5.3 Fair Billing Algorithm

I designed a time-proportional cost-sharing algorithm so that costs are split according to actual participation time rather than divided equally. For example, if Student A used the AC for two hours and Roommate B joined after one hour, the cost is split 2:1 (roughly 67% / 33%) between them, reflecting their actual usage.

5.4 Fault-Tolerant State Machine (ESP32)

I designed a hardware-level state machine with the states BOOTING → STANDBY → SESSION_ACTIVE → FINISHED → FAULT, allowing the firmware to recover predictably from unexpected conditions such as power loss or sensor faults.

5.5 UI/UX Design

The mobile app's UI/UX was designed using Google Stitch, which was used to lay out and iterate on the Dashboard, Session Control, Wallet, and History screens before implementation in React Native.

6. Programming Contribution

6.1 Hardware Firmware (C++)

I wrote approximately 590 lines of ESP32 firmware implementing a BLE GATT server/client, PZEM-004T telemetry acquisition, NVS-based power-cut recovery, LCD display output, emergency-stop button debouncing, and relay control logic.

6.2 Backend REST API (Node.js)

I built more than fifteen API endpoints covering authentication (JWT), wallet management, the full session lifecycle (book, activate, cancel, sync, end), participant management (invite, accept, reject, leave, approve), and push notifications.

6.3 Mobile Application (React Native)

I built five screens — Dashboard, Session Control (with live BLE telemetry), Session History, Wallet, and Profile. BLE connection management was implemented using react-native-ble-plx, and push notification handling was implemented using expo-notifications together with Firebase FCM V1.

7. Tools, Technologies, APIs and Libraries Used

Category	Tool / Technology
Hardware	ESP32-WROOM-32, PZEM-004T v3.0, 5V AC Relay Module
Firmware Language	C++ (Arduino Framework)
Mobile Framework	React Native (Expo SDK 52)
Backend Language	Node.js v22, Express.js
Database	PostgreSQL 15
Authentication	JSON Web Tokens (JWT), bcryptjs
Push Notifications	Firebase Cloud Messaging (FCM V1), expo-notifications
BLE Library	react-native-ble-plx
Firmware Libraries	ArduinoJson, LiquidCrystal_I2C, PZEM004Tv30, ESP32 BLE Arduino
Build Tool	EAS (Expo Application Services)
Version Control	Git
UI/UX Design	Google Stitch

8. Testing Strategies and Reports

8.1 Hardware Testing

- Verified that the relay triggers correctly on START, STOP, and E-STOP commands.
- Tested power-cut recovery by pulling the power plug mid-session and confirming that the session resumed correctly on boot.
- Load-tested with high-draw appliances (an electric iron rated at approximately 1500W) to verify PZEM measurement accuracy.

8.2 Backend API Testing

- Manually tested all API endpoints using Postman.
- Verified that wallet balance deductions are mathematically correct after split billing is applied.
- Verified that database CHECK constraints correctly reject negative kWh values.

8.3 Integration Testing (End-to-End)

- Ran full session flows: Book → BLE Start → Live Telemetry → Auto-finish → Auto-Sync → Wallet Deduction.
- Tested the E-Stop flow: button press → relay off → FINISHED telemetry → app sync → wallet settled.
- Tested the roommate invite flow: invite → FCM push → accept → join → time-based split at settlement.

9. Innovative Contribution

- **Offline-first BLE architecture for IoT billing:** using the student's phone as a secure bridge between offline hardware and the cloud backend is an approach not commonly seen in existing IoT billing systems.
- **NVS-based power-cut recovery for financial data:** persisting billing state to non-volatile flash memory at regular intervals ensures zero financial data loss during power outages.
- **Time-proportional split-billing algorithm:** dynamic cost-splitting based on exact, second-level participation time rather than a simple equal split.

10. Lessons Learnt

- **System design before coding matters:** designing the three-tier offline-first architecture upfront prevented many potential edge cases later in development.
- **Never trust the client:** all financial calculations and validations must be done on the server side, never on the mobile app.
- **Race conditions are real:** I learned to use PostgreSQL ACID transactions and row-level locks to solve real concurrency bugs.
- **Edge hardware is unpredictable:** power-cut scenarios, BLE connection drops, and sensor noise (such as NaN readings from the PZEM sensor) are practical issues rarely covered in textbooks.
- **Security must be a first-class citizen:** financial systems require thinking like an attacker; I fixed multiple loopholes such as a participant leaving a room during an active session to avoid being billed.

References

- React Native Documentation — reactnative.dev/docs/getting-started
- Express.js Documentation — expressjs.com
- PostgreSQL Documentation — postgresql.org/docs
- Espressif ESP32 Datasheet and Arduino BLE Library documentation
- PZEM-004T v3.0 Datasheet
- Relevant IEEE papers on IoT-based smart energy metering (IEEE Xplore)